

Underactuated Humanoid Locomotion

Santiago Restrepo

contact@waajacu.com <https://waajacu.com/>

Learning-based control method for bipedal walking

Abstract—This brief proposes deep reinforcement learning for underactuated humanoid walking. A neural policy learns foot placement, load transfer, and whole-body coordination through interaction with simulated contact dynamics. Joint feedback executes the learned commands. The workflow combines physical modeling, policy training, and validation before hardware deployment; no robot-specific performance is assumed.

Index Terms—Humanoid, bipedal locomotion, underactuation, deep reinforcement learning.

I. MODEL THE FLOATING BASE AND CONTACT

Assume a free-standing humanoid with n actuated joints. Let q denote configuration and v generalized velocity. Between impacts, the floating-base dynamics are [1]

$$\begin{aligned} M(q)\dot{v} + h(q, v) &= B\tau + J_c^T \lambda, \\ \dot{q} &= N(q)v. \end{aligned} \quad (1)$$

M is inertia, h includes gravity and velocity terms, B maps joint torques τ , and J_c^T maps contact forces λ ; N handles orientation kinematics. The base has six unactuated directions. Foot contact changes the available control authority: fixed flat-foot support can be fully actuated in reduced coordinates.

Model unilateral, nonpenetrating contact, friction limits, and impact velocity resets. Foot forces come from the coupled contact dynamics. The policy influences base momentum through joint motion and ground reaction forces; it must learn when to shift weight and place the next foot.

II. CONNECT A NEURAL POLICY TO THE MOTORS

Use command $c = (\text{forward speed, lateral speed, yaw rate})$ and history H of IMU angular velocity, projected gravity, joint positions/velocities, and previous actions. A causal Transformer encodes this history to address partial observability [2].

$$\begin{aligned} a_t &\sim \pi_\theta(\cdot \mid H_t, c_t), \\ q_{\text{des}} &= q_{\text{nom}} + D \tanh(a_t), \\ \tau &= \text{sat}[K_p(q_{\text{des}} - q_a) - K_d v_a]. \end{aligned} \quad (2)$$

θ denotes policy parameters; D scales offsets about q_{nom} , and sat applies motor torque limits. Joint states q_a, v_a drive faster proportional-derivative (PD) loops. Match gains, action rates, and delays to the hardware.

An asymmetric training critic can use simulator ground truth. The actor must use only deployment observations; privileged terrain or velocity signals must not leak into its inputs.

III. TRAIN FOR COMMANDED LOCOMOTION

Train an actor and value critic with proximal policy optimization (PPO) [3] in parallel physics simulations. Collect rollouts, estimate advantages, update the actor with PPO's clipped objective, and fit the critic to return targets. Optimize the expected discounted return:

$$\max_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right]. \quad (3)$$

T is the task horizon and γ the discount factor; bootstrap values at nonterminal rollout truncations. Reward velocity/yaw tracking and upright locomotion; penalize falls, slip, collisions, effort, and abrupt action changes. Normalize terms before tuning weights. Test for shortcuts such as shuffling or hopping when the task calls for walking.

IV. BRIDGE SIMULATION AND HARDWARE

Identify link inertia, joint friction, motor response, and latency. Randomize mass, center of mass, ground friction, actuator strength, sensor noise, and delays over plausible ranges. Add pushes and a curriculum from standing and slow steps to turns and uneven terrain. Such simulation-trained policies have transferred to real humanoids [2]; transfer must be verified for the target robot.

Freeze network weights and observation normalization for deployment. Use the policy mean with the same action transformation and filtering. Enforce actuator and command limits, monitor joint state and temperature, and provide an independent timeout/fall supervisor.

V. MEASURE WHAT GENERALIZES

Evaluate multiple training seeds on held-out terrains, payloads, disturbances, and latency. Compare with a model-based walking baseline using fall rate, command-tracking error, slip, energy per distance, torque saturation, and inference time. Progress from simulation to restrained hardware trials and supervised walking. Report tested conditions and failures; a successful rollout is not a stability proof.

REFERENCES

- [1] R. Tedrake, "Planning and Control through Contact," *Underactuated Robotics*, MIT notes, underactuated.mit.edu/contact.html.
- [2] I. Radosavovic *et al.*, "Real-world humanoid locomotion with reinforcement learning," *Science Robotics*, vol. 9, p. eadi9579, 2024, doi:10.1126/scirobotics.adi9579.
- [3] J. Schulman *et al.*, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.